



University of
Nottingham
UK | CHINA | MALAYSIA

Fundamentals of Artificial Intelligence

Session 5: Adversarial Search | AE1FAI / COMP1037

Dr Qian Zhang | University of Nottingham Ningbo China | Spring



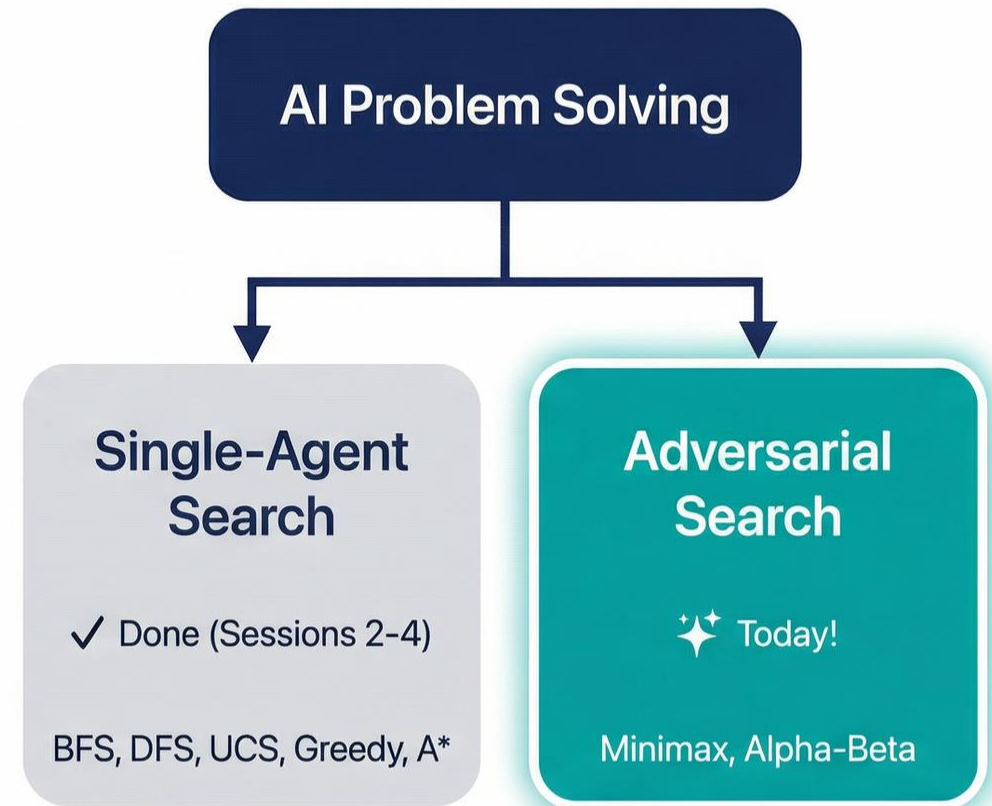


- **Recap: The Story So Far**

- Session 2: Problem Formulation & Representation
- Session 3: Uninformed Search (BFS, DFS, UCS)
- Session 4: Heuristic Search (Greedy, A*)

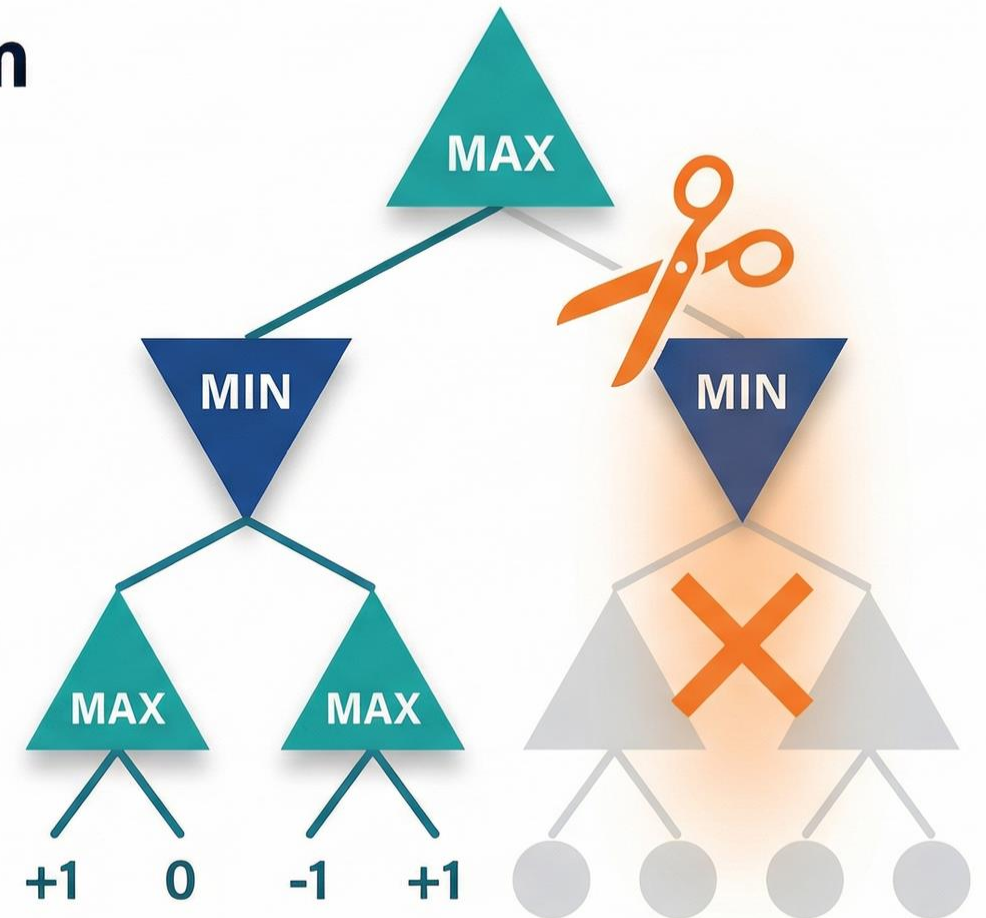
- **Today: Session 5 — Adversarial Search**

- Game playing as a search problem
- Minimax algorithm
- Alpha-beta pruning



Learning Objectives

- Understand the concept of **zero-sum games** and **adversarial search** environments
- Implement the **Minimax algorithm** to find optimal moves in perfect-information games
- Apply **Alpha-Beta pruning** to optimize search efficiency without losing optimality





Game Playing

From single-agent to adversarial search



Why Study Game Playing?

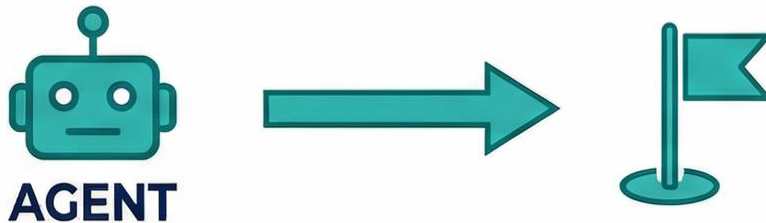
- **Intelligent activity**
easy to measure success
- **Minimal domain knowledge**
- **Solvable by search**



What is Adversarial Search?

Adversarial Search: A search strategy for environments where **TWO AGENTS** compete with **CONFLICTING GOALS**. Each agent tries to maximize its own outcome while minimizing the opponent's.

Single-Agent Search



One agent navigates a **static** environment
Environment does **NOT** fight back



VS

Adversarial Search

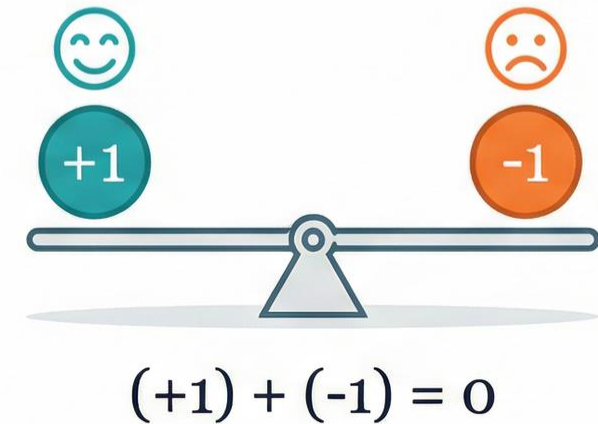


Two agents with **OPPOSITE** objectives
Opponent **ACTIVELY** tries to defeat you



Key difference: In adversarial search, the environment is an intelligent opponent — your search must account for the **WORST** case (opponent plays optimally).

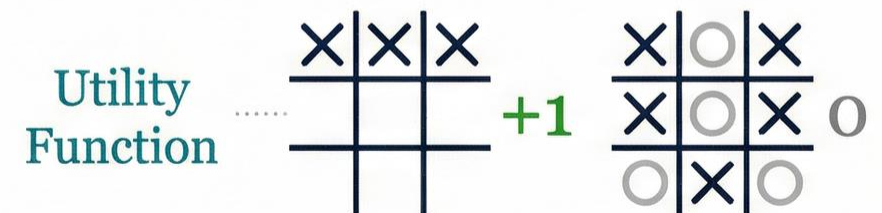
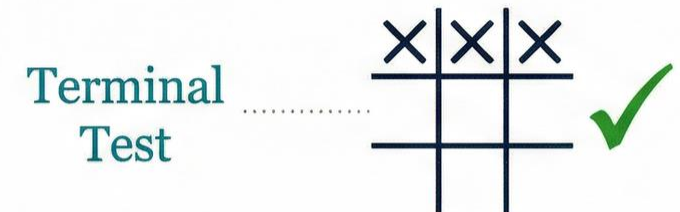
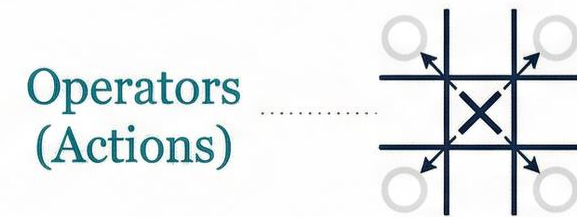
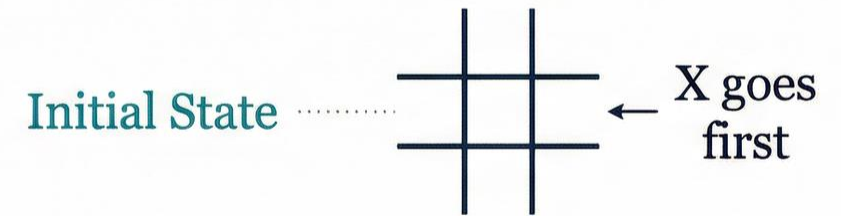
- ◆ Fully observable environments (perfect information) in which two agents act alternately.
- ◆ Utility values at the end of the game are always equal and opposite. (0+1, 1+0, or $\frac{1}{2}+\frac{1}{2}$ — total payoff is always the same)
- ◆ If one player wins a game, the other player necessarily loses.
- ◆ This opposition between the agents' utility functions makes the situation **adversarial**.



Game Search Components

A game can be defined as a kind of search problem with the following components:

- ◆ **Initial State:** board position, indication of whose move it is
- ◆ **Operators (Actions):** define the legal moves that a player can make
- ◆ **Terminal Test:** determines when the game is over (terminal states)
- ◆ **Utility Function:** gives a numeric value for the outcome of a game (e.g., chess: +1, 0, -1)



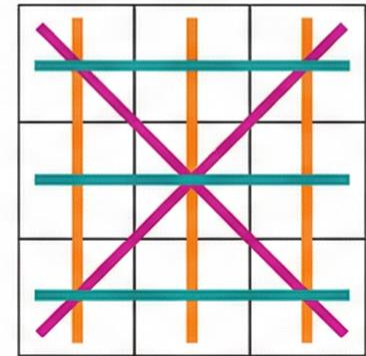
Game Problem in Code

```

1 def make_game_state(board=None, to_move='X'):
2     if board is None:
3         board = ('.',) * 9
4     return {'board': board, 'to_move': to_move}
5
6 def make_game_problem(board=None, to_move='X'):
7     return {
8         'initial': make_game_state(board, to_move),
9         'winning_lines': [
10             (0,1,2), (3,4,5), (6,7,8), # rows
11             (0,3,6), (1,4,7), (2,5,8), # cols
12             (0,4,8), (2,4,6), # diags
13         ]
14     }

```

winning_lines



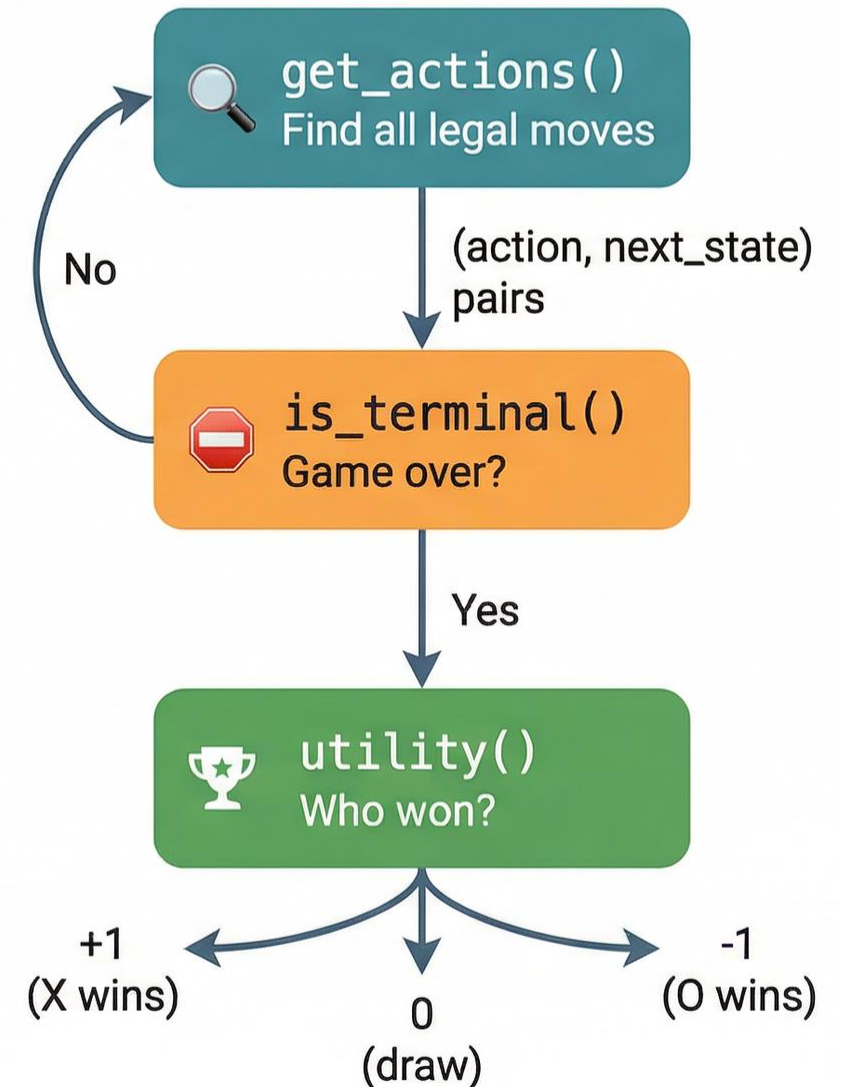
initial state

board:	('.,',) * 9
to_move:	'X'

BOTTOM RIGHT
State = board position + whose turn

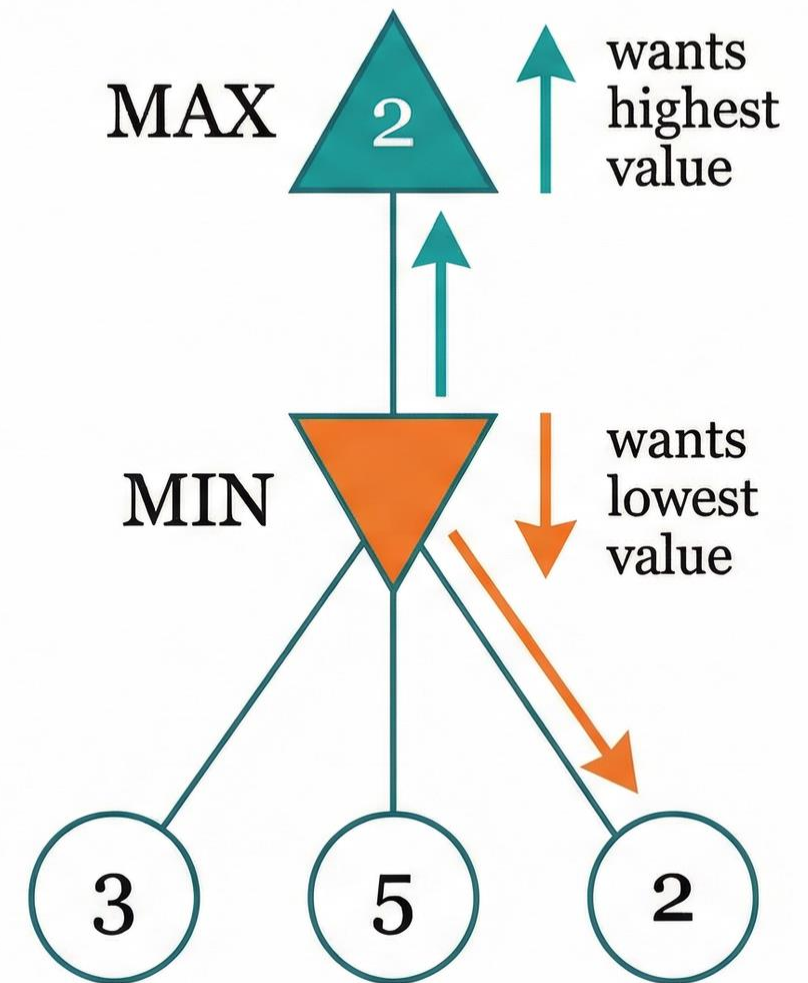
Core Game Functions

```
def get_actions(problem, state):  
    actions = []  
    for pos, cell in enumerate(state['board']):  
        if cell == '.':  
            next_state = apply_move(state, pos)  
            actions.append((pos, next_state))  
    return actions  
  
def is_terminal(problem, state):  
    return check_winner(...) is not None \  
        or '.' not in state['board']  
  
def utility(problem, state):  
    winner = check_winner(problem, state['board'])  
    if winner == 'X': return +1  
    if winner == 'O': return -1  
    return 0
```



A strategy for finding the optimal move in a two-player zero-sum game.

- ◆ Origin: John von Neumann (1944)
- ◆ Core Idea: **Maximise** your position whilst **minimising** your opponent's
- ◆ **Utility Function**: measures how good a position is — called the **minimax value**
- ◆ Assumption: Both players play **optimally** (perfect play)



How Minimax Values Are Computed

How Minimax Values Are Computed

Terminal nodes: $\text{minimax}(n) = \text{utility}(n)$

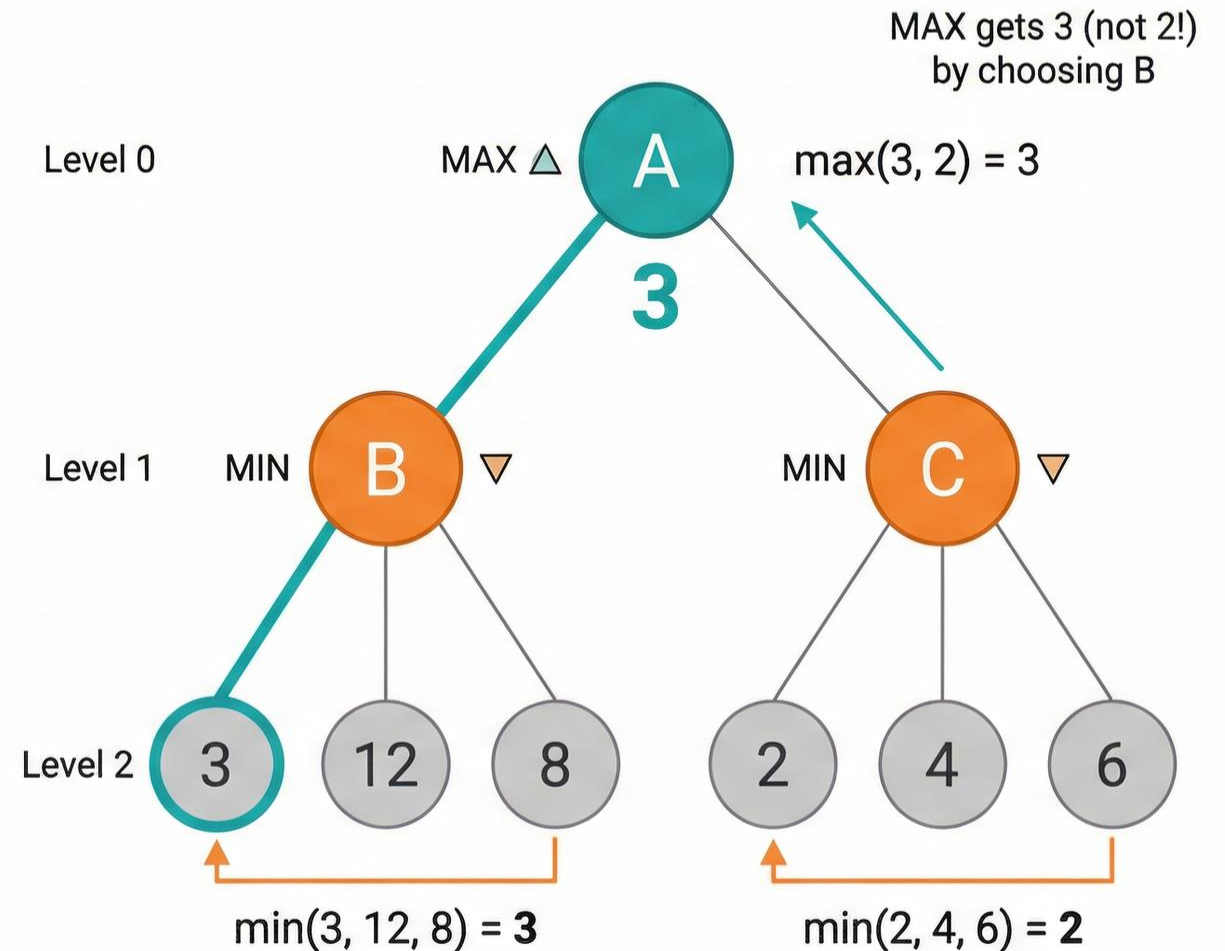
The exact value at game end

MAX nodes:
 $\text{minimax}(n) = \max(\text{minimax}(c) \text{ for } c \text{ in children}(n))$

Pick the HIGHEST child value 

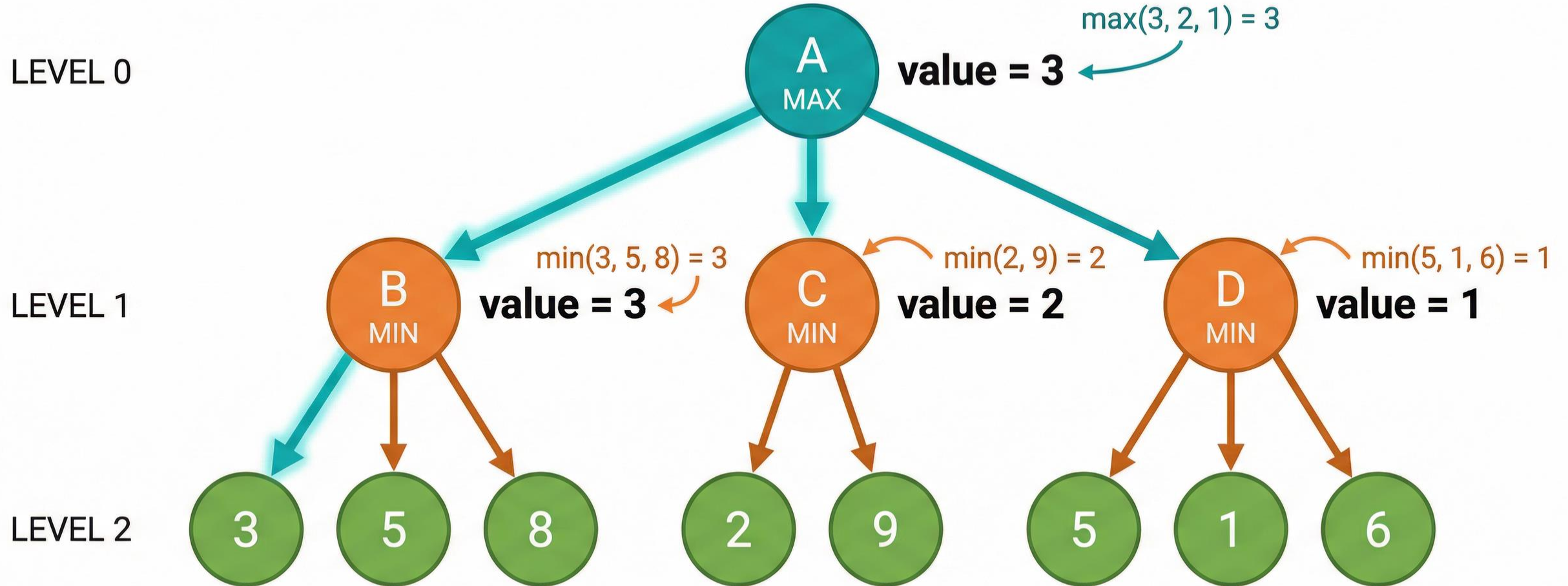
MIN nodes:
 $\text{minimax}(n) = \min(\text{minimax}(c) \text{ for } c \text{ in children}(n))$

Pick the LOWEST child value 





Minimax: Visual Trace



Step 1: Evaluate leaves
(terminal utility)

Step 2: MIN picks
minimum child

Step 3: MAX picks
maximum child

Step 4: Best move = B
(value 3)

Nim Game: Problem Formulation

Rules

Start with 7 tokens. Each turn, divide one pile **into two non-empty, non-equal piles**. The player who **cannot move loses**.

Formulate as Game Search:



Initial State: One pile of 7 tokens, MAX moves first



Actions: Split any pile (size ≥ 3) into two non-empty, non-equal parts

Example: 7 can split into (1,6), (2,5), or (3,4)

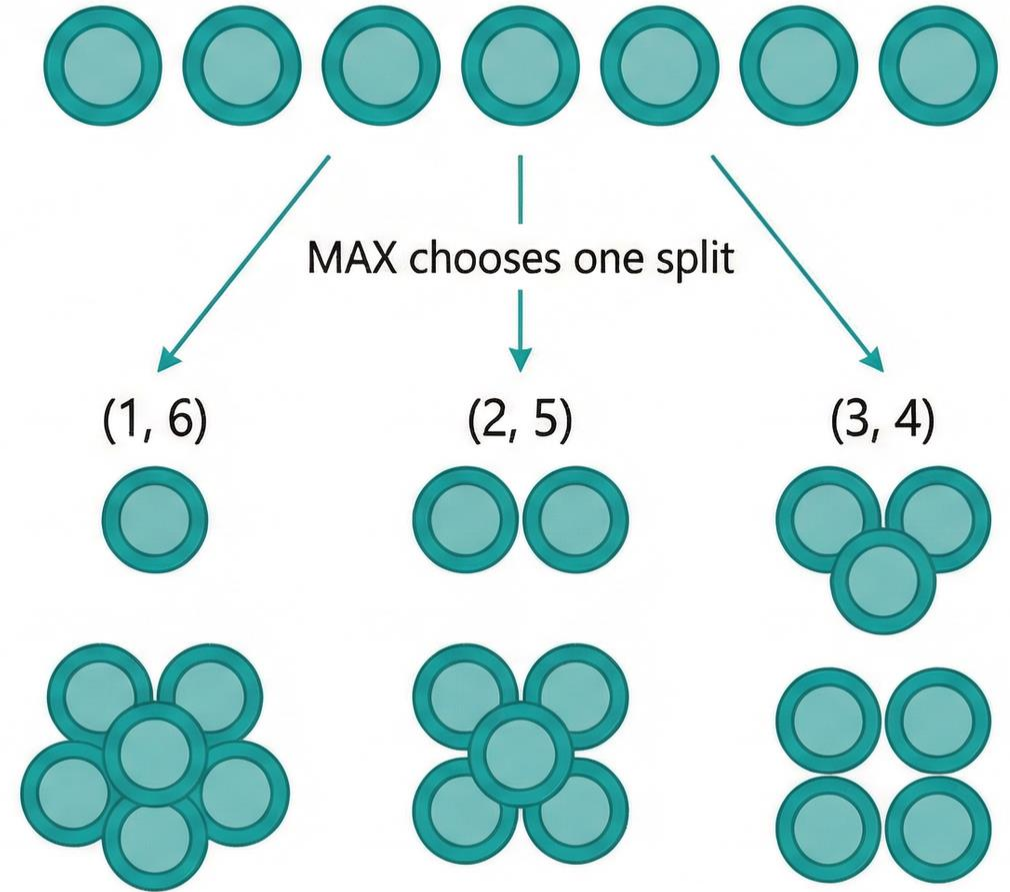


Terminal Test: No pile has ≥ 3 tokens (no valid splits)



Utility: Player who cannot move **LOSES**

- MAX cannot move = 0 (MIN wins)
- MIN cannot move = 1 (MAX wins)





Nim Game: Building the Game Tree

Rule: Pick a pile with ≥ 3 tokens, split into 2 non-empty, non-equal piles. Player who cannot move loses.

Level 0 (MAX)

[7]

Level 1 (MIN)

[6,1]

[5,2]

[4,3]

Level 2 (MAX)

[5,1,1]

[4,2,1]

[3,2,2]

[3,3,1]

Level 3 (MIN)

[4,1,1,1]

[3,2,1,1]

[2,2,2,1]

MIN's turn, No pile ≥ 3

Level 4 (MAX)

[3,1,1,1,1]

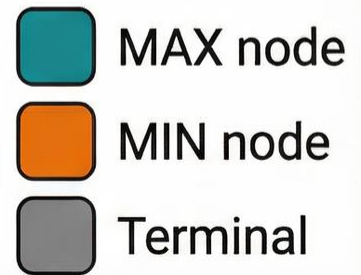
[2,2,1,1,1]

MAX's turn, No pile ≥ 3

Level 5 (MIN)

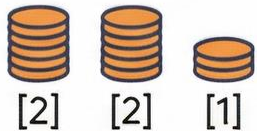
[2,1,1,1,1,1]

MIN's turn, No pile ≥ 3

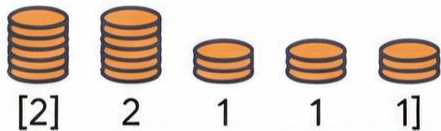


Assigning Terminal Utilities

✓ If it is MIN's turn and MIN cannot move:
utility = 1 (MAX wins!)



✗ If it is MAX's turn and MAX cannot move:
utility = 0 (MIN wins!)



Key insight: A pile can only be split if it has ≥ 3 tokens.
When ALL piles have 1 or 2 tokens, the game is OVER.

Level 3
(MIN)

[2,2,2,1]

1
utility = 1
(MIN stuck)

Level 4
(MAX)

[2,2,1,1,1]

0
utility = 0
(MAX stuck)

Level 5
(MIN)

[2,1,1,1,1,1]

1
utility = 1
(MIN stuck)



Nim Game: Building the Game Tree

Rule: Pick a pile with ≥ 3 tokens, split into 2 non-empty, non-equal piles. Player who cannot move loses.

Level 0 (MAX)

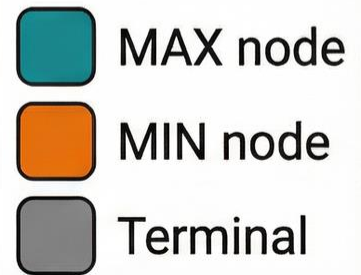
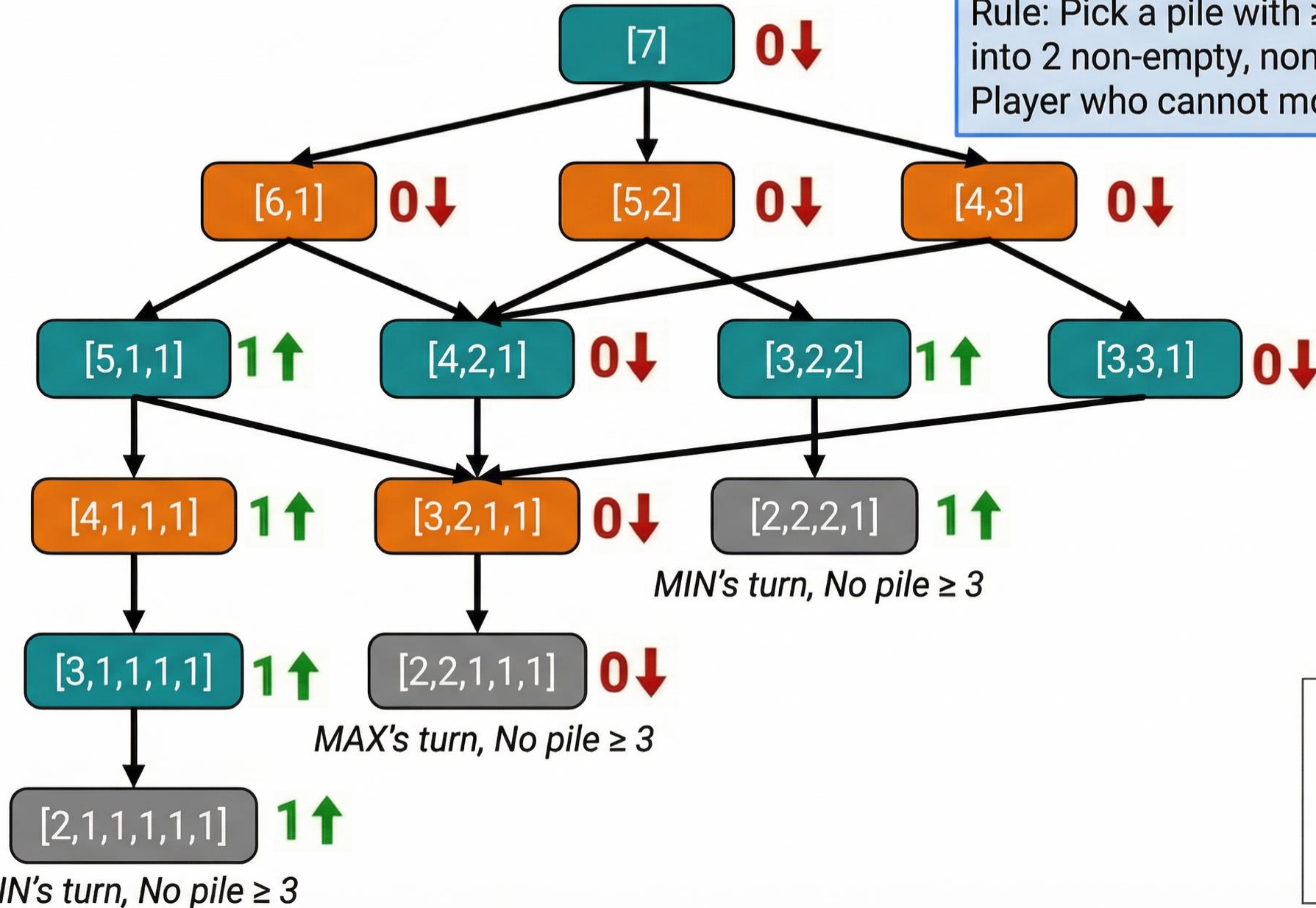
Level 1 (MIN)

Level 2 (MAX)

Level 3 (MIN)

Level 4 (MAX)

Level 5 (MIN)



Minimax: Python Implementation

```
1 def minimax(problem, state, is_maximizing):
2     if is_terminal(problem, state):
3         return utility(problem, state), None
4
5     if is_maximizing:
6         best_value = -float('inf')
7     else:
8         best_value = +float('inf')
9
10    best_move = None
11    for move, next_state in get_actions(...):
12        value, _ = minimax(problem, next_state,
13                           not is_maximizing)
14        if is_maximizing and value > best_value:
15            best_value = value
16            best_move = move
17        if not is_maximizing and value < best_value:
18            best_value = value
19            best_move = move
20    return best_value, best_move
```



1. BASE CASE: Game over?
Return utility score directly.



2. INIT: MAX starts at $-\infty$,
MIN starts at $+\infty$ (any real
value beats these)



3. RECURSE: Try each move,
ask opponent for THEIR best.
Flip is_maximizing!



4. UPDATE: MAX keeps
highest value, MIN keeps
lowest value found so far.

Adversarial Search - Interactive Demo

Session 5 | COMP1037 Fundamentals of AI

1. Nim Game Tree

2. Pile-Split Nim

3. Minimax Search

4. Alpha-Beta Pruning

5. Play Nim vs AI

Tree: Textbook (4 levels)

Next Step →

Auto Play ▶

Reset

Step 32/53 C: min so far = 2

MAX

MIN

MAX

- MAX node
- MIN node
- Currently evaluating
- Value computed
- Not yet visited

Minimax Execution Log

[32] C: min so far = 2
[31] F = max(children) = 2
[30] F: max so far = 2
[29] Leaf 1 = 1
[28] Visit 1 (MIN)
[27] F: max so far = 2
[26] Leaf 2 = 2
[25] Visit 2 (MIN)
[24] Visit F (MAX)
[23] Visit C (MIN)
[22] A: max so far = 17
[21] B = min(children) = 17
[20] B: min so far = 17
[19] E = max(children) = 20
[18] E: max so far = 20
[17] Leaf 8 = 8
[16] Visit 8 (MIN)
[15] E: max so far = 20
[14] Leaf 20 = 20
[13] Visit 20 (MIN)
[12] Visit E (MAX)
[11] B: min so far = 17
[10] D = max(children) = 17
[9] D: max so far = 17
[8] Leaf 17 = 17
[7] Visit 17 (MIN)
[6] D: max so far = 3
[5] Leaf 3 = 3
[4] Visit 3 (MIN)
[3] Visit D (MAX)
[2] Visit B (MIN)
[1] Visit A (MAX)

Statistics

Nodes evaluated: 12
Root value: -
Best move: -

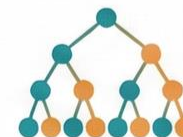
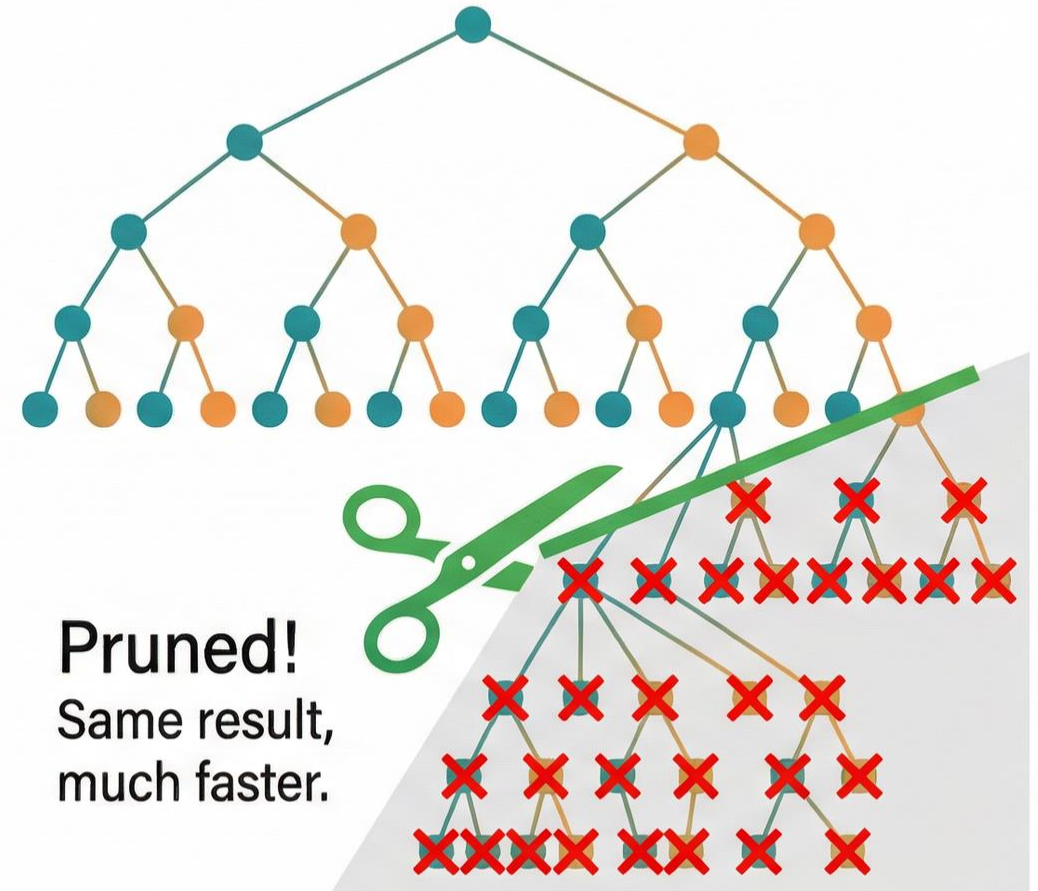
Why We Need Alpha-Beta Pruning

The Problem with Minimax

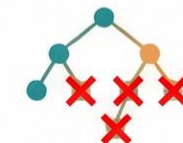
- ◆ Game trees grow exponentially: $O(b \text{ to the power } d)$ where b = branching factor, d = depth
- ◆ Even Nim(7) has about 20 nodes — manageable
- ◆ But real games are enormous:
Chess: b is about 35, d is about 80
Go: b is about 250, d is about 150
- ◆ Evaluating every node is too slow!

Solution: Alpha-Beta Pruning

Compute the correct minimax decision WITHOUT looking at every node in the game tree.
Prune branches that cannot affect the final decision.



Minimax:
explores ALL
nodes



Alpha-Beta: skips
irrelevant branches

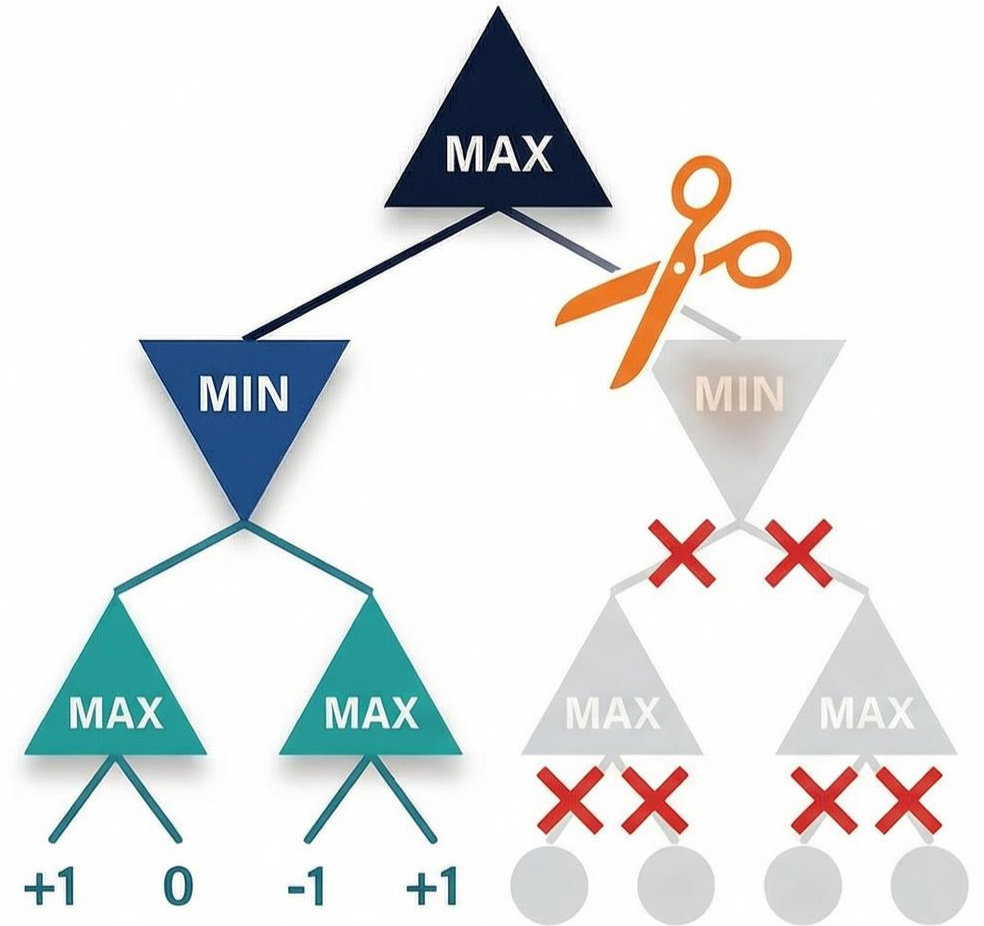


Alpha-Beta Pruning

Smarter search through pruning

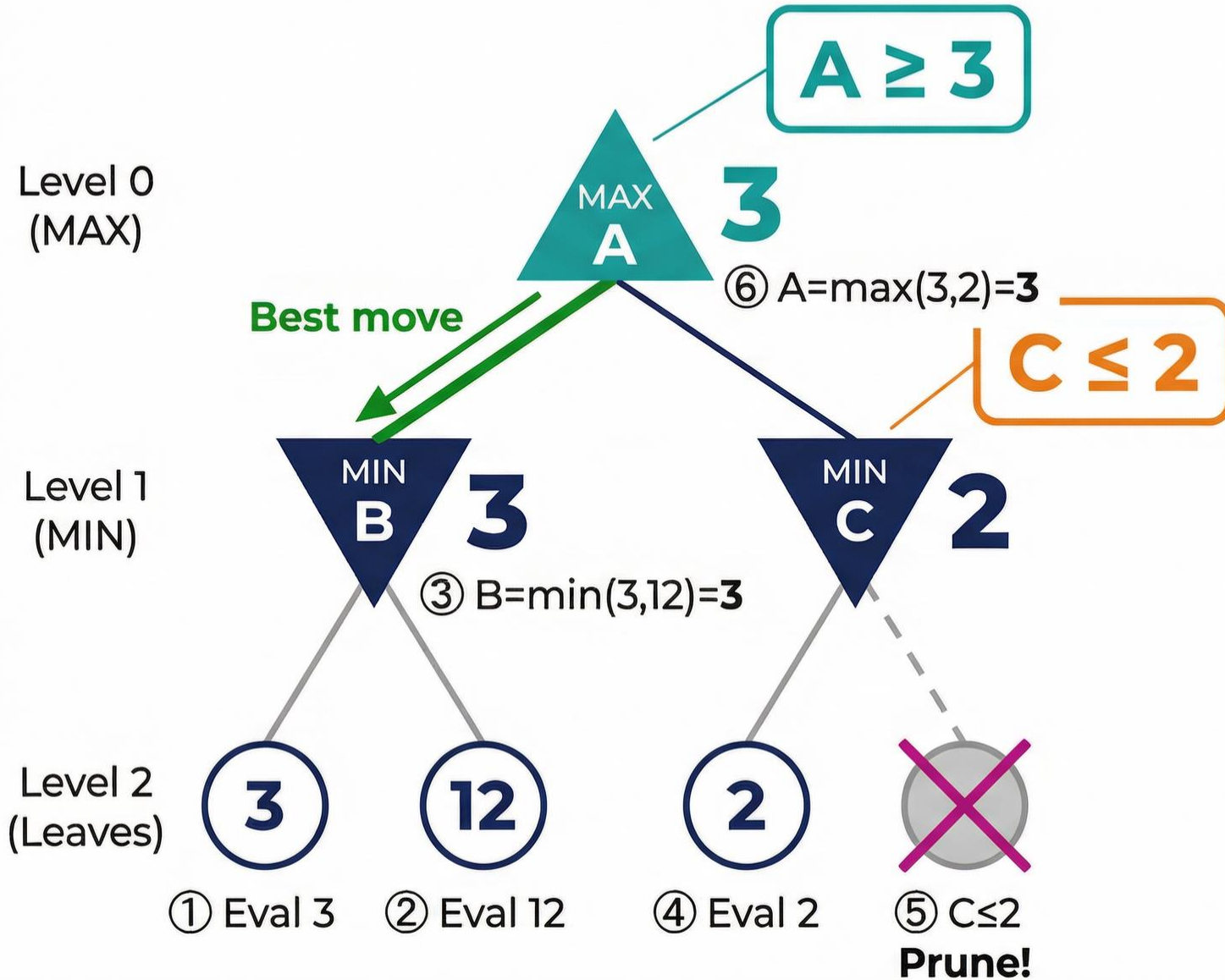
Why Alpha-Beta Pruning?

- Game trees **grow exponentially**
- Many branches are **irrelevant**
- Same answer, **much faster**





Alpha-Beta: Step by Step



Alpha Pruning Logic

1. Left subtree (B) evaluated
A already knows it can get ≥ 3

2. C finds first child = 2
C is MIN, so $C \leq 2$

3. Key comparison

$$3 > 2$$

MAX won't choose C!
PRUNE remaining children ✂

Alpha Cutoff: at a MIN node, if value $\leq$ alpha of ancestor MAX, prune.

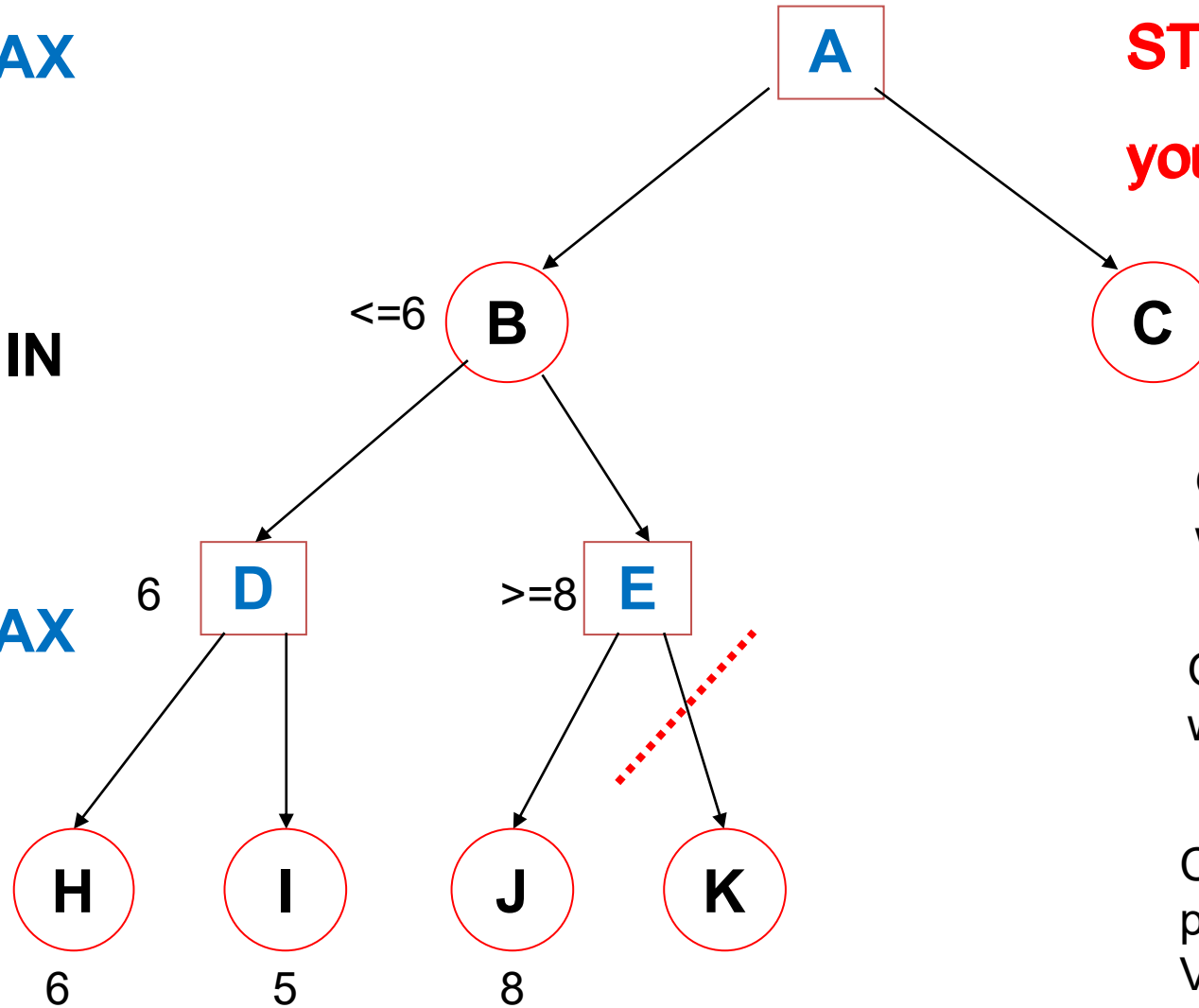


Alpha-Beta: Step by Step

MAX

MIN

MAX



**STOP! What else can
you deduce now!?**

On discovering $\text{util}(D) = 6$
we know that $\text{util}(B) \leq 6$

On discovering $\text{util}(J) = 8$
we know that $\text{util}(E) \geq 8$

Can stop expansion of E as best
play will not go via E
Value of K is irrelevant – prune it!

 = agent

 = opponent

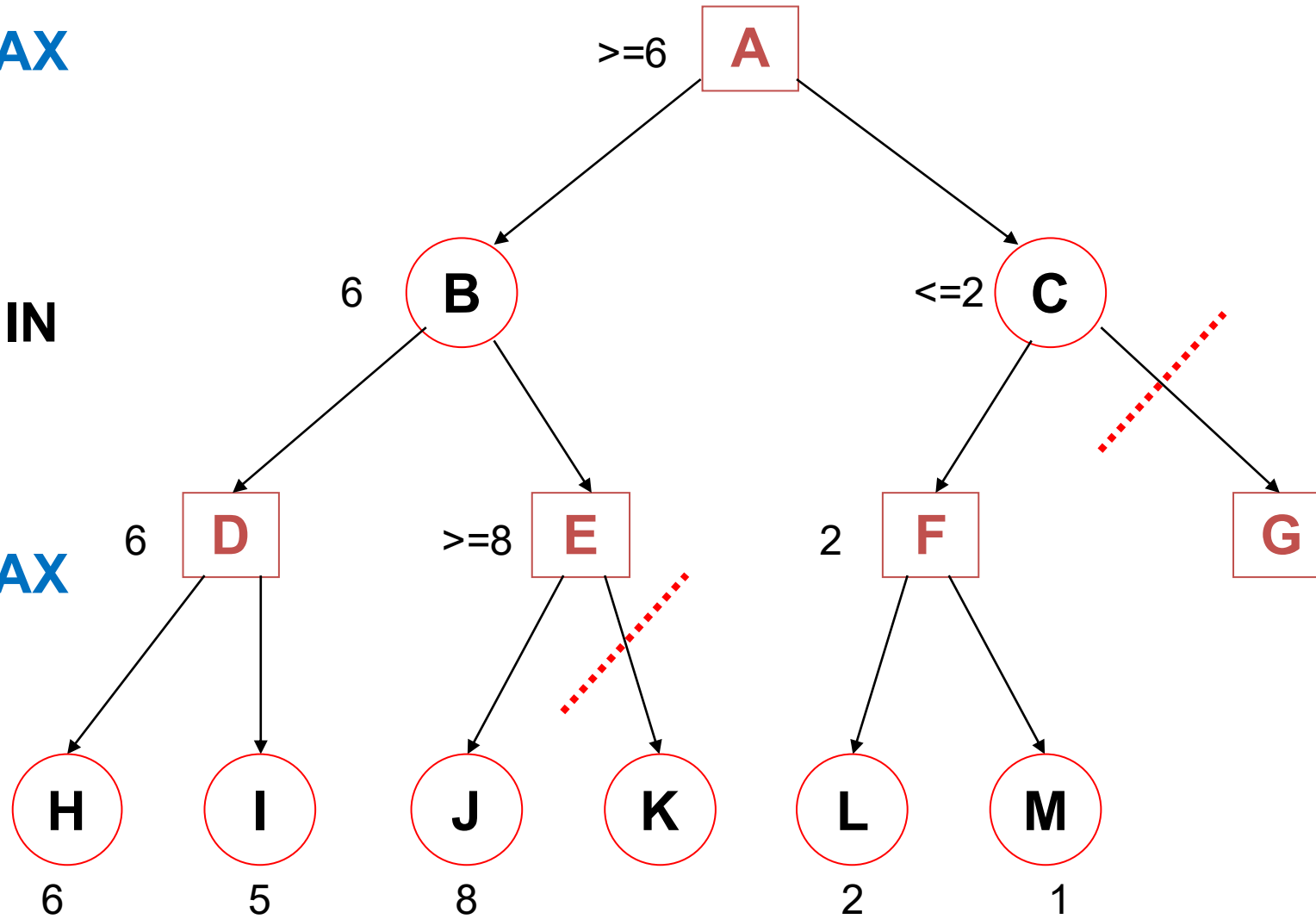


Alpha-Beta: Step by Step

MAX

MIN

MAX



 = agent

 = opponent

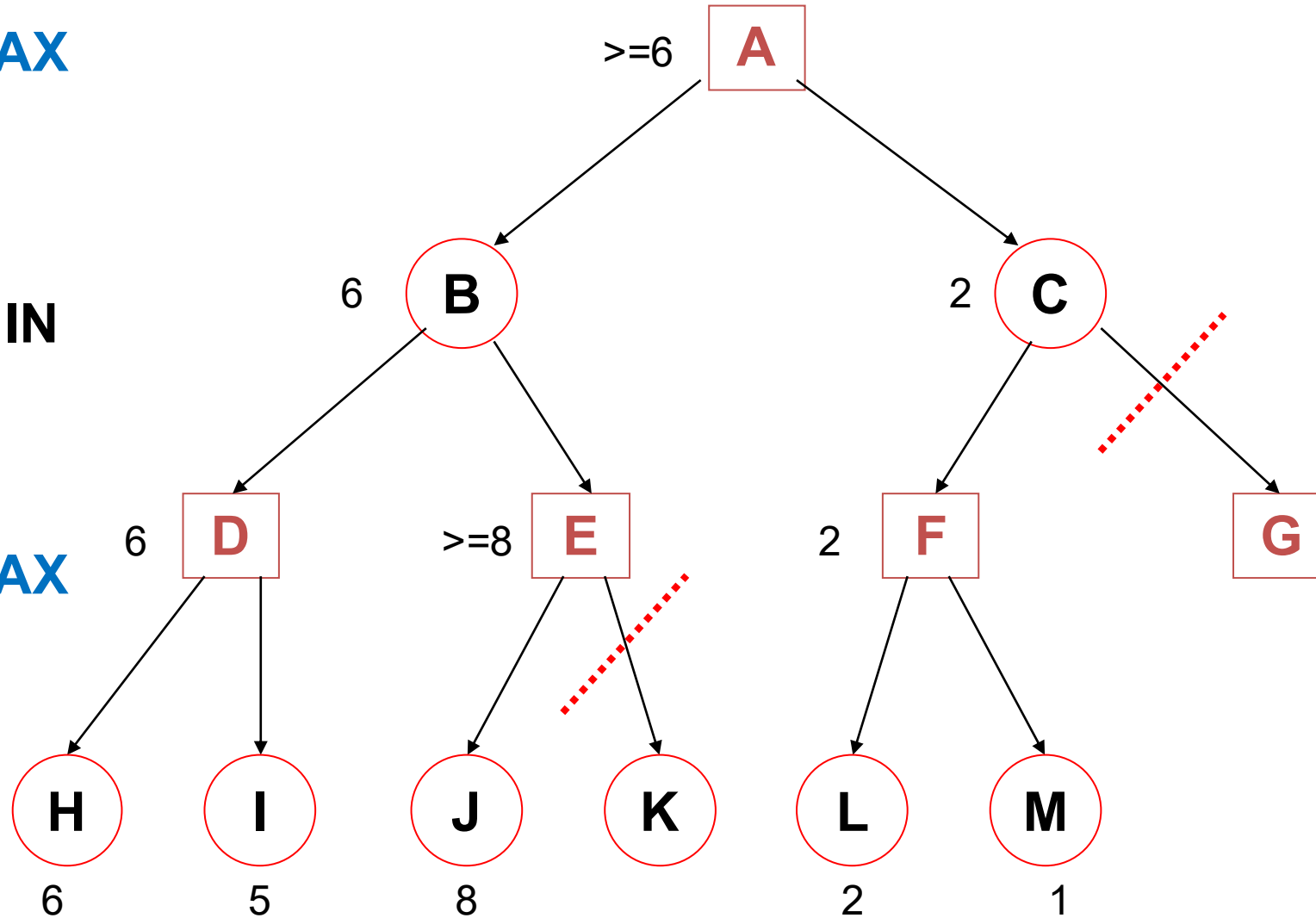


Alpha-Beta: Step by Step

MAX

MIN

MAX



 = agent

 = opponent

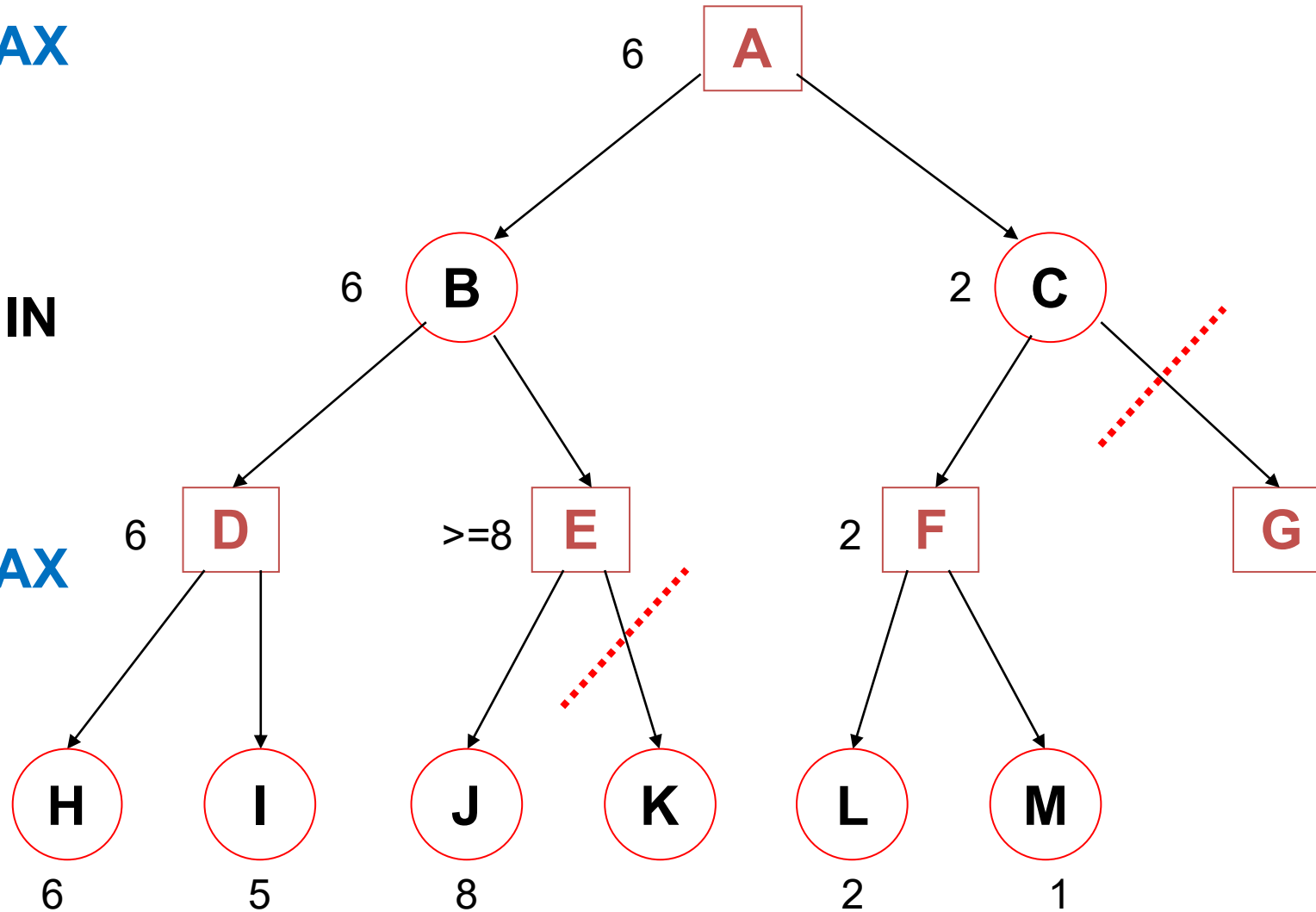


Alpha-Beta: Step by Step

MAX

MIN

MAX



= agent

= opponent

Alpha-beta Pruning



Alpha and Beta Parameters

α
alpha

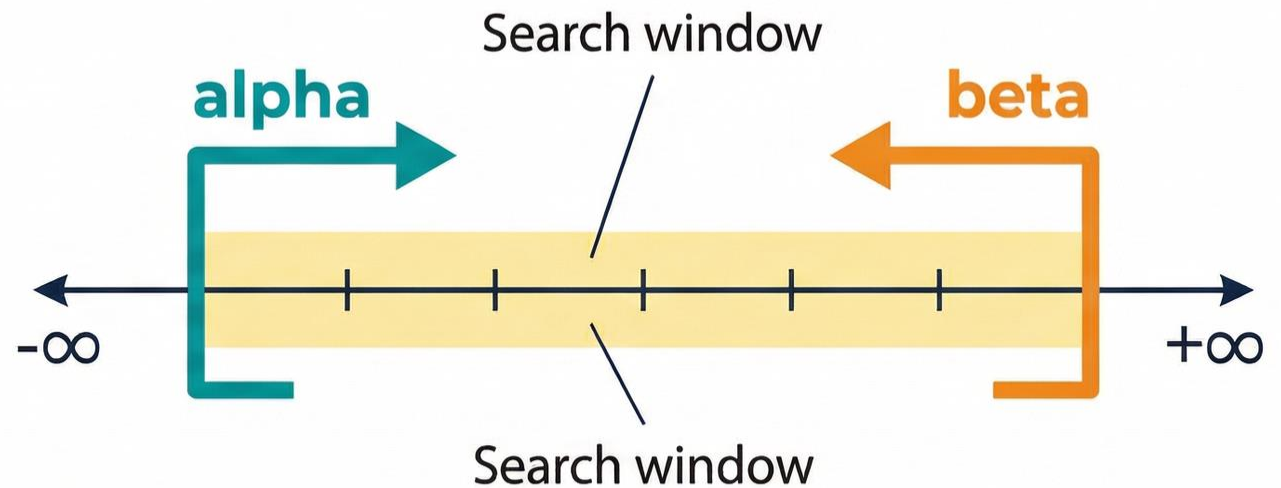
Best value MAX can guarantee

Starts at negative infinity
Updated at MAX nodes

β
beta

Best value MIN can guarantee

Starts at positive infinity
Updated at MIN nodes



**Prune when $\alpha \geq \beta$
(window closes!)**

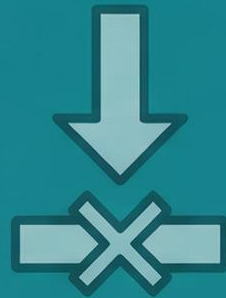


Alpha-Beta Cutoff Rules

Beta Cutoff

At MAX node:
value $\geq$ beta

MIN will never choose this path



Alpha Cutoff

At MIN node: value $\leq$ alpha

MAX will never choose this path



Requires DFS (not BFS!)

Pruning depends on evaluating
subtrees first

Best-first ordering maximizes pruning

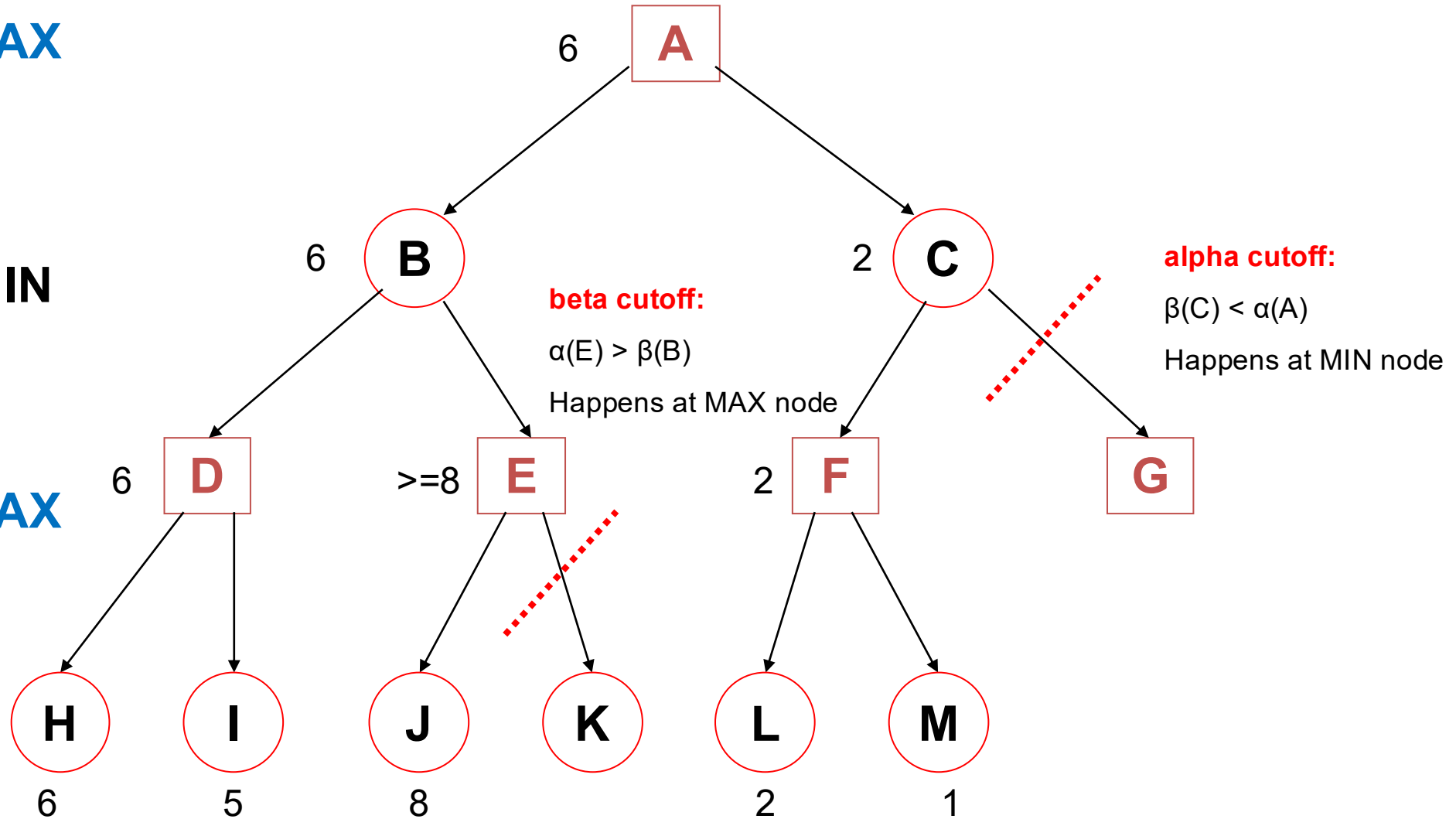


Alpha-Beta: Step by Step

MAX

MIN

MAX



 = agent

 = opponent

Alpha-beta Pruning

Alpha-Beta: Python Implementation

```
1 def alphabeta(problem, state,
2               alpha, beta, is_maximizing):
3     if is_terminal(problem, state):
4         return utility(problem, state), None
5     if is_maximizing:
6         best_value = -float('inf')
7     else:
8         best_value = +float('inf')
9     best_move = None
10    for move, next_state in get_actions(...):
11        value, _ = alphabeta(problem, next_state,
12                             alpha, beta, not is_maximizing)
13        if is_maximizing and value > best_value:
14            best_value = value
15            best_move = move
16        alpha = max(alpha, best_value)
17        if not is_maximizing and value < best_value:
18            best_value = value
19            best_move = move
20        beta = min(beta, best_value)
21        if alpha >= beta:
22            break
23    return best_value, best_move
```

NEW: MAX raises alpha floor.
alpha = max(alpha, best_value).
“I can guarantee at least this much.”

NEW: MIN lowers beta ceiling.
beta = min(beta, best_value).
“I can limit you to at most this much.”



PRUNE! When alpha $\geq$ beta,
the search window has
closed. No need to explore
remaining children. break!

Only 3 lines added to minimax = massive speedup!

Adversarial Search - Interactive Demo

Session 5 | COMP1037 Fundamentals of AI

1. Nim Game Tree

2. Pile-Split Nim

3. Minimax Search

4. Alpha-Beta Pruning

5. Play Nim vs AI

Tree: Simple (pruning demo) Next Step → Auto Play ▶ Reset

Step 19/19 A final = 3

MAX

MIN

- MAX node
- MIN node
- Currently evaluating
- Value computed
- Pruned

3
 $\alpha=3$
 $\beta=+\infty$
A

3
 $\alpha=-\infty$
 $\beta=3$
B

2
 $\alpha=3$
 $\beta=2$
C

12
val=12

2
val=2

8
val=8

4
val=14

Alpha-Beta Window

Current node: A (MAX)
 α (MAX floor): 3
 β (MIN ceiling): $+\infty$
Window: (3, $+\infty$)
Decision: -

Alpha-Beta Execution Log

- [19] A final = 3
- [18] A: value=3, $\alpha=3$, $\beta=+\infty$
- [17] C final = 2
- [16] PRUNE 14: $\beta(2) \leq \alpha(3) \rightarrow \alpha$ cutoff!
- [15] PRUNE 8: $\beta(2) \leq \alpha(3) \rightarrow \alpha$ cutoff!
- [14] C: value=2, $\alpha=3$, $\beta=2$
- [13] Leaf 2 = 2
- [12] Visit 2 (MAX) [$\alpha=3$, $\beta=+\infty$]
- [11] Visit C (MIN) [$\alpha=3$, $\beta=+\infty$]
- [10] A: value=3, $\alpha=3$, $\beta=+\infty$
- [9] B final = 3
- [8] B: value=3, $\alpha=-\infty$, $\beta=3$
- [7] Leaf 12 = 12
- [6] Visit 12 (MAX) [$\alpha=-\infty$, $\beta=3$]
- [5] B: value=3, $\alpha=-\infty$, $\beta=3$
- [4] Leaf 3 = 3
- [3] Visit 3 (MAX) [$\alpha=-\infty$, $\beta=+\infty$]
- [2] Visit B (MIN) [$\alpha=-\infty$, $\beta=+\infty$]
- [1] Visit A (MAX) [$\alpha=-\infty$, $\beta=+\infty$]

Statistics

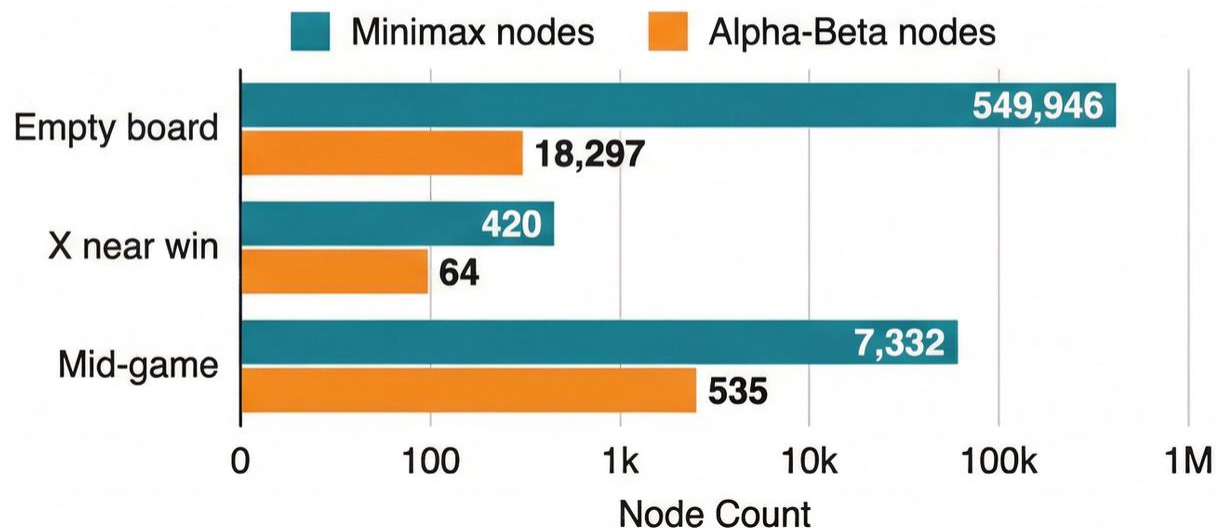
Nodes evaluated: 6
Nodes pruned: 2
Root value: 3
Savings vs Minimax: 2 nodes saved (25%)



Minimax vs Alpha-Beta: Results

Position	Minimax Nodes	Alpha-Beta Nodes	Speedup	Same Move?
Empty board	549,946	18,297	30.1x	Yes (✓)
X near win	420	64	6.6x	Yes (✓)
Mid-game	7,332	535	13.7x	Yes (✓)

Nodes Evaluated



Same move, far fewer nodes!

- Best case: $O(b^{d/2})$ vs $O(b^d)$
- Up to 30x faster on empty Tic-Tac-Toe board
- Alpha-Beta is strictly better than Minimax

 [Try the interactive demo!](#)

- **Adversarial search**
games as search problems
- **Minimax**
optimal play, perfect opponent
- **Alpha-Beta pruning**
same answer, less work

Reading: ALMA Chapter 5 (5.1-5.3)

